



JASHORE UNIVERSITY OF SCIENCE AND TECHNOLOGY

Department of Computer Science and Engineering

LAB REPORT

Course Code: CSE 3101

Course Title: Operating System and System Programming Lab

Lab Report No: 08

Lab Title: Implementation of Banker's Algorithm for Finding Safe Sequence of Processes

Submitted To:

Monishanker Halder,
Assistant Professor,
Dept. of CSE, JUST

Submitted By:

Name: Adnan Zaman Niloy
Student ID: 210142
Session: 2021-2022
Year/Semester: 3rd Year, 1st Semester

Date of Submission: 21th July, 2025

Experiment Title: Implementation of Banker's Algorithm for Finding Safe Sequence of Processes

1. Objective

The primary objective of this laboratory experiment is to implement the Banker's Algorithm, a deadlock avoidance algorithm used in operating systems to determine if a system is in a safe state and to find a safe sequence of process execution that avoids deadlock situations.

2. Theory

2.1 Deadlock

Deadlock is a situation in multiprogramming where two or more processes are unable to proceed because each is waiting for resources held by the other processes. Four necessary conditions for deadlock are:

- Mutual Exclusion
- Hold and Wait
- No Preemption
- Circular Wait

2.2 Banker's Algorithm

The Banker's Algorithm, developed by Edsger Dijkstra, is a deadlock avoidance algorithm that simulates resource allocation to determine if granting a resource request would lead the system into an unsafe state. The algorithm maintains several data structures:

- **Allocation Matrix (Alloc):** Resources currently allocated to each process
- **Maximum Matrix (Max):** Maximum resource requirements for each process
- **Available Vector (Avail):** Currently available resources in the system
- **Need Matrix (Need):** Remaining resource requirements for each process ($\text{Need} = \text{Max} - \text{Alloc}$)

2.3 Safe State

A system is in a safe state if there exists a sequence of process execution where all processes can complete their execution without causing deadlock. This sequence is called a "safe sequence."

3. Algorithm Description

The Banker's Algorithm follows these steps:

1. **Initialize:** Calculate the Need matrix ($\text{Need}[i][j] = \text{Max}[i][j] - \text{Alloc}[i][j]$)
2. **Find Process:** Search for a process P_i such that:
 - P_i is not finished ($f[i] = 0$)

- $\text{Need}[i] \leq \text{Available}$
- 3. **Simulate Completion:** If such a process is found:
 - Add P_i to the safe sequence
 - Release P_i 's resources ($\text{Available} = \text{Available} + \text{Alloc}[i]$)
 - Mark P_i as finished ($f[i] = 1$)
- 4. **Repeat:** Continue until all processes are finished or no suitable process is found
- 5. **Check Result:** If all processes are finished, the system is in a safe state

4. Source Code Implementation

The implementation uses C++ and includes the following key components:

4.1 Data Structures

- $\text{alloc}[n][m]$: Allocation matrix
- $\text{max}[n][m]$: Maximum requirement matrix
- $\text{avail}[m]$: Available resources vector
- $\text{need}[n][m]$: Need matrix (calculated)
- $\text{safeseq}[n]$: Safe sequence array
- $f[n]$: Finish status vector

4.2 Source Code

```
#include <bits/stdc++.h>
using namespace std;
int main()
{
    int n, m; // n = number of processes, m = number of resources
    cin >> n >> m;
    int alloc[n][m], max[n][m], avail[m];

    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            cin >> alloc[i][j];
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            cin >> max[i][j];
    for (int i = 0; i < m; i++)
        cin >> avail[i];

    int safeseq[n], idx = 0;
    vector<int> f(n, 0);
    int need[n][m];
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
```

```

        need[i][j] = max[i][j] - alloc[i][j];

for (int k = 0; k < n; k++)
{
    for (int i = 0; i < n; i++)
    {
        if (f[i] == 0)
        {
            int flag = 0;
            for (int j = 0; j < m; j++)
            {
                if (need[i][j] > avail[j])
                {
                    flag = 1;
                    break;
                }
            }
            if (flag == 0)
            {
                safeseq[idx] = i;
                idx++;
                for (int y = 0; y < m; y++)
                    avail[y] += alloc[i][y];
                f[i] = 1;
            }
        }
    }
}

for (int i = 0; i < n; i++)
{
    if (f[i] == 0)
    {
        cout << "The system is not in a safe state.\n";
        return 0;
    }
}

cout << "The system is in a safe state.\nSafe sequence is: ";
for (int i = 0; i < n - 1; i++)
    cout << "P" << safeseq[i] << " -> ";
cout << "P" << safeseq[n - 1] << endl;
return 0;}

```

5. Test Cases and Results

5.1 Test Case 1 (Safe State)

Input Parameters:

- Processes: 5, Resources: 3
- Allocation Matrix:

0 1 0

2 0 0

3 0 2

2 1 1

0 0 2

- Maximum Matrix:

7 5 3

3 2 2

9 0 2

2 2 2

4 3 3

- Available Resources: [3, 3, 2]

Need Matrix (Calculated):

7 4 3

1 2 2

6 0 0

0 1 1

4 3 1

Output:

The system is in a safe state.

Safe sequence is: P1 -> P3 -> P4 -> P0 -> P2

Analysis: The system successfully found a safe sequence, indicating that all processes can complete execution without deadlock.

5.2 Test Case 2 (Unsafe State)

Input Parameters:

- Same process and resource configuration as Test Case 1
- Modified Maximum Matrix:

` 7 5 3

3 2 2

9 0 2

4 2 2 $\leftarrow$ Modified

5 3 3 $\leftarrow$ Modified

- Available Resources: [0, 0, 0] $\leftarrow$ Modified

Output:

The system is not in a safe state.

Analysis: With reduced available resources and increased maximum requirements, the algorithm cannot find a safe sequence, indicating potential deadlock.

6. Algorithm Trace for Test Case 1

Step	Process	Need Check	Available Before	Available After	Safe Sequence
1	P1	$[1,2,2] \leq [3,3,2]$ ✓	[3,3,2]	[5,3,2]	P1
2	P3	$[0,1,1] \leq [5,3,2]$ ✓	[5,3,2]	[7,4,3]	P1→P3
3	P4	$[4,3,1] \leq [7,4,3]$ ✓	[7,4,3]	[7,4,5]	P1→P3→P4
4	P0	$[7,4,3] \leq [7,4,5]$ ✓	[7,4,5]	[7,5,5]	P1→P3→P4→P0
5	P2	$[6,0,0] \leq [7,5,5]$ ✓	[7,5,5]	[10,5,7]	P1→P3→P4→P0→P2

7. Time and Space Complexity Analysis

7.1 Time Complexity

- **Best Case:** $O(n^2m)$ - when processes can be scheduled in order
- **Worst Case:** $O(n^2m)$ - where n is the number of processes and m is the number of resources

7.2 Space Complexity

- **Space Complexity:** $O(nm)$ for storing matrices (allocation, maximum, need)
- Additional $O(n + m)$ for vectors (available, finished, safe sequence)

8. Conclusion

The Banker's Algorithm is a crucial deadlock avoidance technique in operating systems. This lab successfully implemented the algorithm to determine system safety and generate a safe execution sequence if possible. The test cases confirmed the correctness of the implementation, demonstrating both safe and unsafe scenarios.